

OMERO GYM

Class Booking System

Theme: Appointment & Capacity Booking System

Final Project Report, Phase 3 (PHP & MySQL Integration)

Name	Index No.	Registration Number
M R M Ashfak	2696	ITT/2024/011
M A M Ahsan	2694	ITT/2024/009

GitHub Repository: <https://github.com/itt2024009/omerogym.git>

Submission Date : 2026.09.27

Rajarata University of Sri Lanka
Department of ICT
ICT 1209 - Web Technologies

ContentAs

List of Figures.....	1
1. Project Overview	2
1.1. Purpose	2
1.2. Target Audience.....	2
1.3. Key Features	2
2. System Design	3
2.1. Database Schema	3
2.2. Admin Access	3
2.3. Folder Structure	4
3. Implementation	4
3.1 User Authentication & Roles	4
3.2 Session Protection.....	6
3.3 Class Catalog and Booking	6
3.4 My Workouts Dashboard	7
3.5 Contact Form	7
3.6 Admin Control Panel	8
3.7. Security Measures	10
4. Challenges and Solutions	10
4.1. Keeping the original frontend untouched	10
4.2. Protecting pages that are plain HTML files.....	10
4.3. Adding a contact form that did not exist yet	10
4.4. Fitting the required table structure to a booking app	10
5. Individual Contribution.....	11
6. Setup Instructions	11
7. References	12

List of Figures

- Figure 1: Member Login Page
- Figure 2: Member Registration Page
- Figure 3: Book a Training Slot Page
- Figure 4: My Workouts Dashboard
- Figure 5: Contact Form
- Figure 6: Admin Control Panel Dashboard
- Figure 7: Workout Plans and Pricing Management
- Figure 8: Trainer Management
- Figure 9: Messages Management

1. Project Overview

1.1. Purpose

OMERO GYM is a web based platform for booking training sessions and fitness classes. Members can browse the available sessions, choose a suitable time, and confirm a booking through the website without having to contact the gym staff manually. In Phase 3, the PHP and MySQL backend was added to the Phase 2 frontend. This allows user accounts, session details, and bookings to be stored in the database instead of only being kept in the browser.

The project was developed with three main goals:

- Authentication: Members must log in before they can access the booking pages. This prevents users who are not signed in from making bookings.
- Booking control: The system checks the selected date and time before confirming a booking. This helps reduce overcrowding and duplicate bookings.
- Member control: Members can use the My Workouts dashboard to view upcoming bookings, check previous sessions, and cancel upcoming bookings when needed.

1.2. Target Audience

- Gym members and athletes can plan their workouts around available time periods and reserve places in group classes.
- Personal trainers and instructors can manage their training sessions and keep track of the members assigned to them.
- Gym administrators can monitor bookings, manage sessions and trainers, and keep track of the overall scheduling activity.

1.3. Key Features

- Responsive multi-page site (desktop, tablet, mobile) built with HTML5, CSS3, and vanilla JavaScript.
- Member registration and login with bcrypt-hashed passwords, PHP sessions, and role based redirects (member vs admin).
- Live class/service catalog loaded from MySQL, with category filtering (Gym Floor Access, Group Fitness Classes, Personal Training).
- Slot booking flow (date → time block → confirmation) saved to MySQL, with assigned trainers looked up automatically.
- Personal dashboard (“My Workouts”) listing upcoming and past bookings, with cancellation capabilities.
- Contact form for enquiries, stored directly in the database.
- Admin control panel, manage classes/services and trainers, view all session reservations, and read contact messages.
- The system uses server side validation and PDO prepared statements together with client side JavaScript validation.

2. System Design

2.1. Database Schema

The omero_gym database contains six tables created by database.sql: users, messages, trainers, sessions_catalog, admins, and bookings. The database also includes starter records for sessions, trainers, and the administrator account.

Table	Key Fields	Purpose
users	id, name, email (unique), phone, password, created_at	Member accounts used for registration and login.
messages	id, name, email, message, created_at	Enquiries submitted through the contact form.
trainers	id, name, specialty, available, created_at	Trainer details and their booking availability.
sessions_catalog	id, session_key (unique), category, title, description, price, duration, created_at	The available workout and training sessions shown to members and managed by the administrator.
admins	id, name, email (unique), password, created_at	Administrator accounts kept separate from regular member accounts.
bookings	id, user_id (FK), session_id, title, price, duration, trainer_id (FK), trainer_name, booking_date, booking_time, status, created_at	Booking records containing the selected session, trainer, date, time, price, duration, and status.

reservations maintains foreign keys referencing users, classes_services, and trainers. Deleting a user cascades to their reservations; deleting a trainer sets any reservation's trainer_id to NULL to preserve transaction records.

2.2. Admin Access

The database includes a separate admins table for administrator accounts. The admin setup page can be used once to create an administrator account with a bcrypt hashed password. After login, the administrator is taken to the admin control panel. Regular users register through the member registration page and are treated as members.

2.3. Folder Structure

OMERO-GYM/

- |— includes/
 - | |— db.php PDO connection to the omero_gym database
 - | |— functions.php shared helpers (session guard, JSON helpers, sanitising)
- |— auth/
 - | |— register.php creates a user, hashes password, starts session
 - | |— login.php verifies credentials, starts session
 - | |— logout.php destroys the session
 - | |— check.php used by app.js to confirm a member is logged in
- |— bookings/
 - | |— create.php saves a new booking for the logged-in member (+ trainer)
 - | |— list.php returns the member's bookings (+ trainer)
 - | |— cancel.php deletes a booking (only if it belongs to the member)
- |— sessions.php public: returns the workout-plan catalog (admin-managed)
- |— trainers.php public: returns trainers + availability (admin-managed)
- |— contact.php Messages / contact form + handler → messages table
- |— admin/ admin control panel (signs in via the shared login.html)
 - | |— logout.php / seed_admin.php (login.php just forwards to ../login.html)
 - | |— dashboard.php, sessions.php, trainers.php, messages.php
 - | |— includes/ admin session guard + shared header/footer
- |— index.php redirects to classes.html or login.html
- |— dashboard.php redirects to workouts.html (the member dashboard)
- |— database.sql creates the database + all tables (see below)
- |— app.js frontend logic — fetches catalog/trainers from the DB
- |— *.html / *.css frontend pages and styling

The frontend pages (login.html, register.html, classes.html, book.html, workouts.html) and their CSS were kept exactly as designed in Phase 2. Only app.js was extended so its existing forms and buttons call the new PHP endpoints with fetch() instead of saving to localStorage.

3. Implementation

3.1 User Authentication & Roles

The register and login pages keep the original responsive design. When a user submits a form, app.js sends the required data to the PHP authentication files. Passwords are stored using password_hash()

with PASSWORD_BCRYPT and checked using password_verify(). The session ID is also regenerated after successful authentication.

The screenshot shows the 'OMERO GYM PORTAL' login interface. At the top, there is a navigation bar with the 'OMERO GYM' logo on the left and 'Login' and 'Create Account' buttons on the right. The main content area features a central login form with a red 'Login' button. The form includes fields for 'Email Address' (containing 'athlete@example.com') and 'Password' (masked with '*****'). Below the password field is a 'Sign In' button. A link 'New to Omero GYM? Create an Account Here' is positioned at the bottom of the form. The background is dark with the text 'DISCIPLINE TODAY STRENGTH TOMORROW' repeated on both sides.

Figure 1: Member Login Page

The screenshot shows the 'CREATE ACCOUNT' registration interface. The top navigation bar is identical to the login page. The central form is titled 'CREATE ACCOUNT' and includes a sub-header 'Join us to instantly unlock interactive capacity slots'. The form contains fields for 'Full Name' (filled with 'John Doe'), 'Email Address' (filled with 'john@example.com'), 'Phone Number' (filled with '+84 7X XXX XXXX'), and 'Password' (with a note 'Minimum 6 characters' and 'Passwords are hashed securely using bcrypt in our system.'). A red 'Register Now' button is at the bottom of the form, with a link 'Already registered? Login Here' below it. The background features the same dark theme and 'DISCIPLINE TODAY STRENGTH TOMORROW' text as the login page.

Figure 2: Member Registration Page

3.2 Session Protection

Pages like `classes.html`, `book.html`, and `workouts.html` are restricted to members. On page load, `app.js` triggers a check to `auth/check.php`; unauthenticated users are immediately redirected to `login.html`. Every admin route enforces authentication via `admin/includes/admin_auth.php`, rendering administrative panels inaccessible to standard member accounts

3.3 Class Catalog and Booking

`classes.html` dynamically fetches available offerings from `classes/list.php` (backed by `classes_services` and `trainers`). Selecting "Book Slot" leads to `book.html`, where members specify a date and time slot. Form confirmation dispatches data to `bookings/create.php`, which validates availability and inserts a reservation linked to the authenticated user ID.

Figure 3: Book a Training Slot Page

3.4 My Workouts Dashboard

workouts.html queries bookings/list.php, retrieving user-specific reservations categorized into "Upcoming" and "Past" sessions based on server date comparison. Cancellation requests pass through bookings/cancel.php, ensuring users can only modify their own reservations.

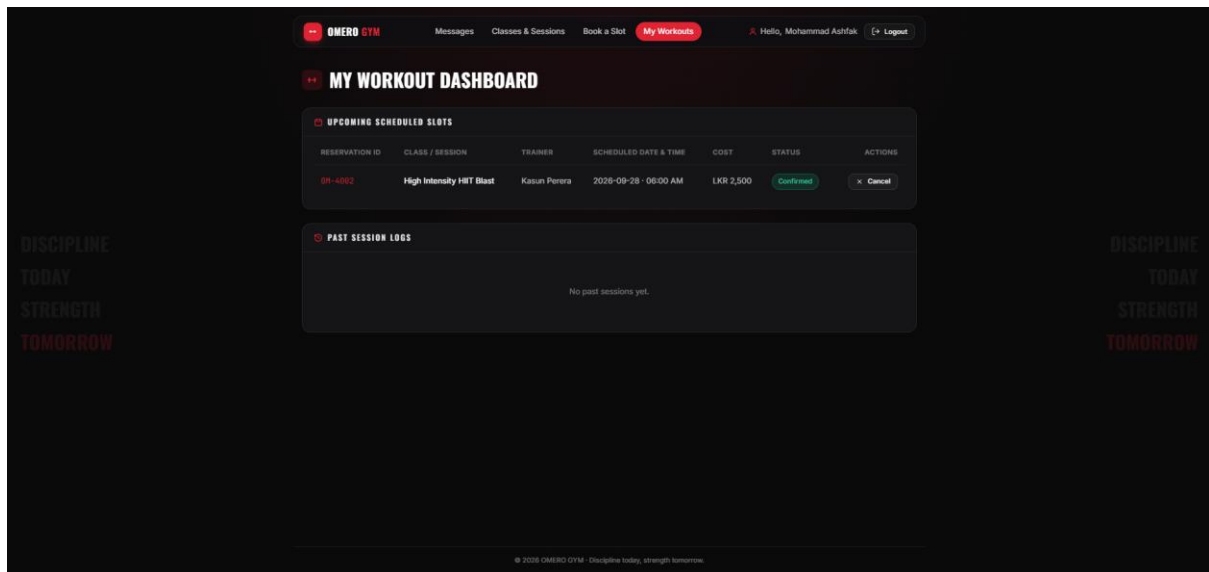


Figure 4: My Workouts Dashboard

3.5 Contact Form

contact.php provides an inquiry submission interface styled with the dark OMERO GYM theme using common.css and login.css[cite: 2, 3]. Client-side JS and server side PHP validate all form inputs before storing inquiries in the messages table

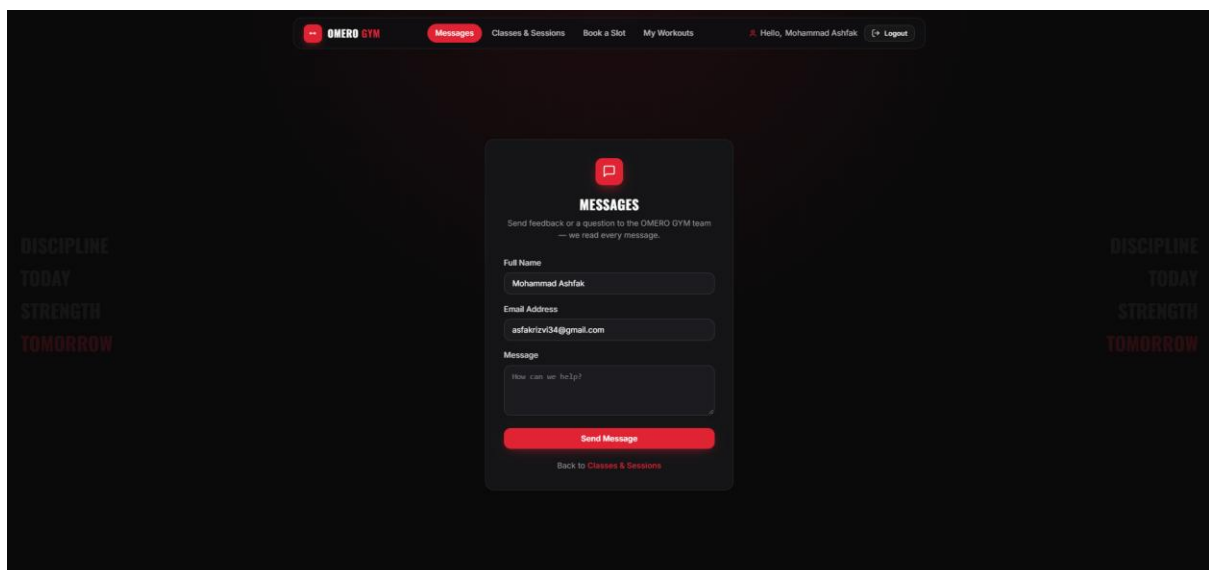


Figure 5: Contact Form

3.6 Admin Control Panel

The admin/ workspace contains server-rendered PHP modules utilizing admin/admin.css. admin/dashboard.php aggregates key analytics including account counts, total reservations, and recent bookings. admin/sessions.php, admin/trainers.php, and admin/messages.php provide management tools for class sessions, staff directories, and member communications.

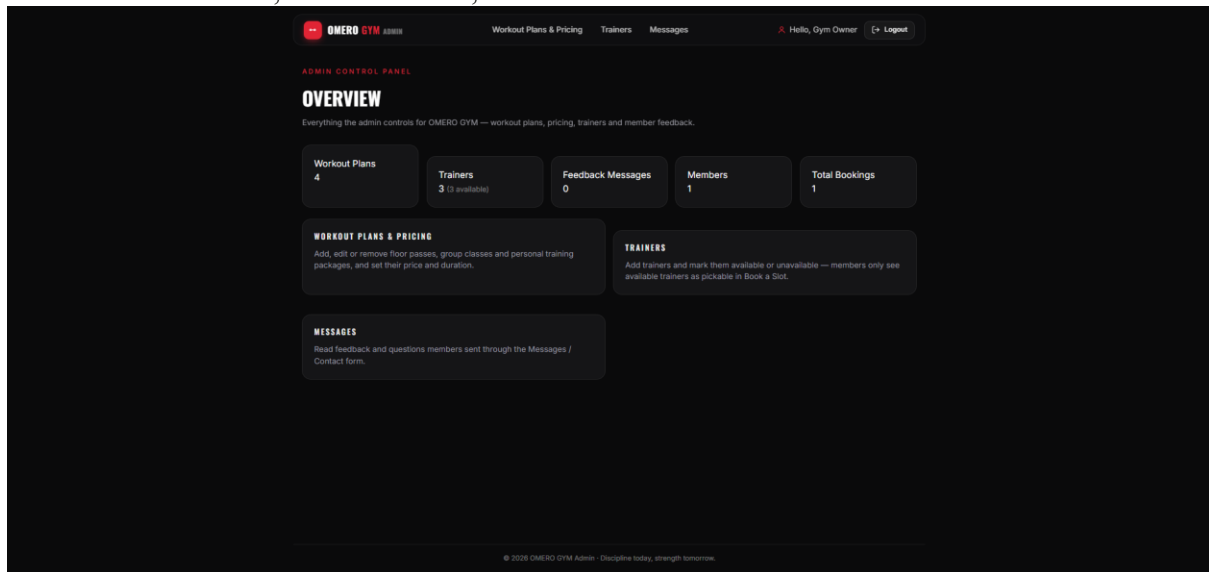


Figure 6: Admin Control Panel Dashboard

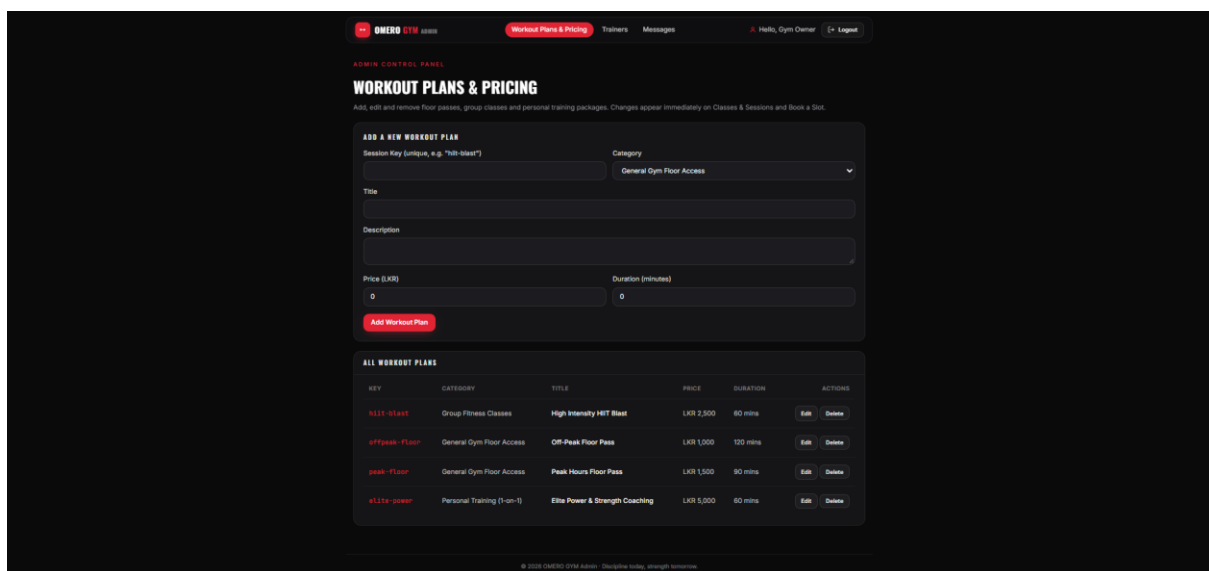


Figure 7: Workout Plans and Pricing Management

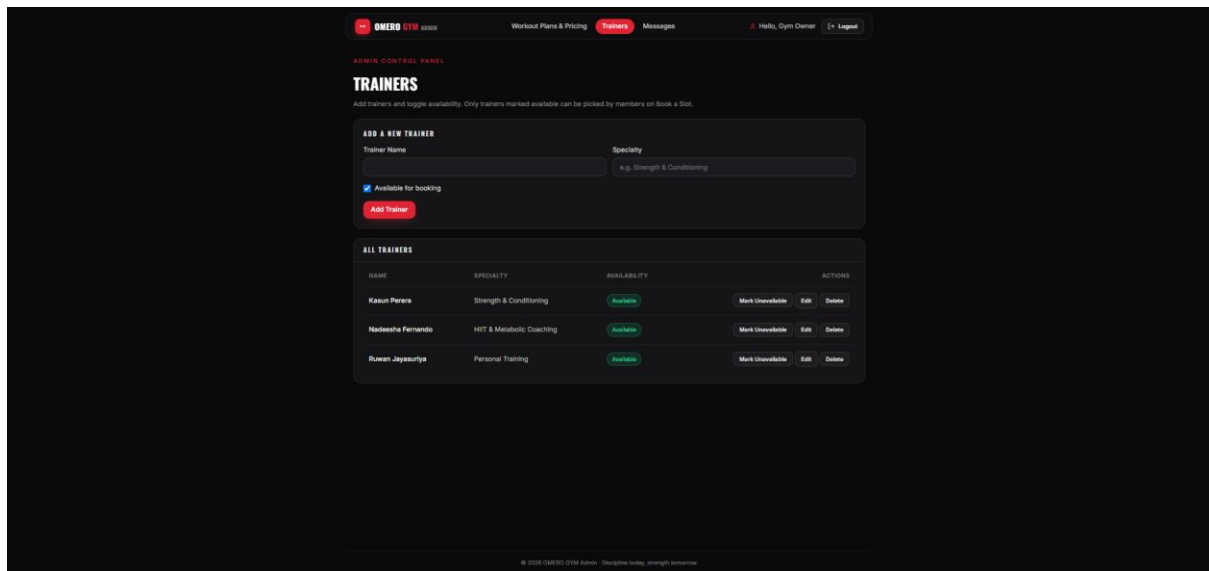


Figure 8: Trainer Management

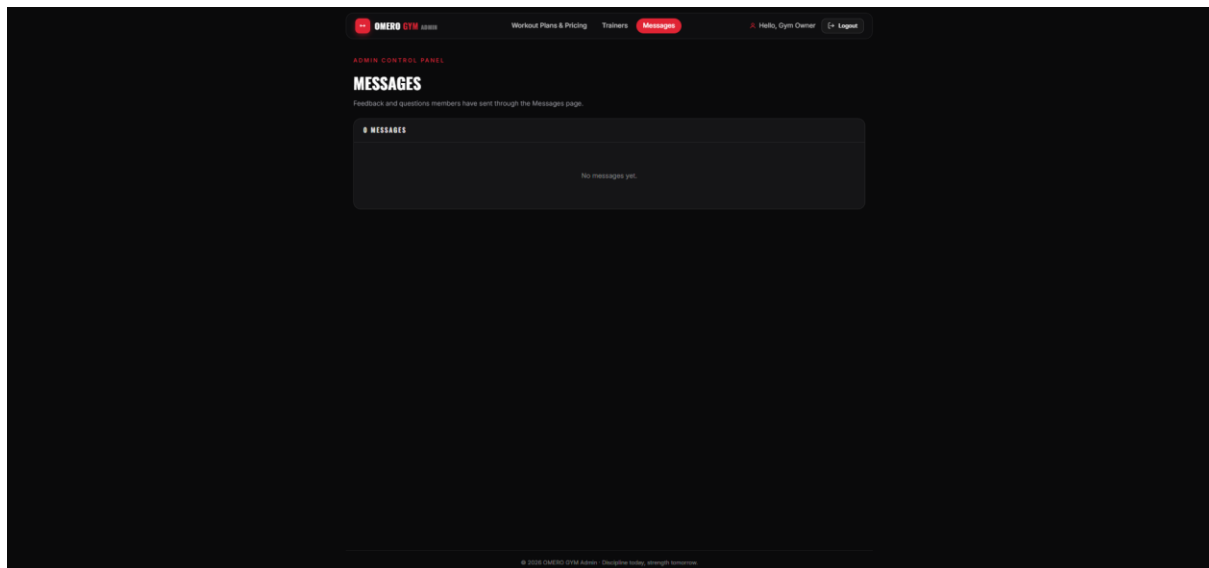


Figure 9: Messages Management

3.7. Security Measures

- Prepared Statements: All database queries utilize PDO prepared statements to prevent SQL Injection.
- Password Hashing: Plaintext passwords are never saved; strong bcrypt hashing is enforced.
- Session Hijacking Prevention: `session_regenerate_id(true)` resets session identities upon authentication state changes.
- Access Control: User roles (member vs admin) are strictly validated on the server for all endpoint actions.
- XSS Prevention: Input data is sanitized and safely escaped using `htmlspecialchars()` prior to rendering.

4. Challenges and Solutions

4.1. Keeping the original frontend untouched

The Phase 2 frontend stored user and booking information in the browser's `localStorage` and did not have a PHP backend. The main challenge was to connect the existing frontend to real authentication and a MySQL database without changing its visual design. The solution was to update `app.js` so that the existing forms and buttons communicate with the PHP endpoints through `fetch()`.

4.2. Protecting pages that are plain HTML files

The pages `classes.html`, `book.html`, and `workouts.html` are static HTML files, so PHP session checks cannot be placed directly inside those pages. To handle this, `app.js` checks `auth/check.php` when each page loads. If the session is not valid, the user is redirected to `login.html`.

4.3. Adding a contact form that did not exist yet

The Phase 3 requirements include a contact form and a messages table, but the original frontend did not have a contact page. The contact page was therefore created using the same colours, fonts, and form styling as the existing website so that it fits naturally with the rest of the system.

4.4. Fitting the required table structure to a booking app

The assignment example uses a workout table with fields such as activity, duration, and calories burned. OMERO GYM is designed around session booking rather than workout logging, so the main theme table in this project is bookings. It stores the selected session, price, duration, trainer, date, time, and booking status.

5. Individual Contribution

Name	Registration Number	Contribution
M R M Ashfak	ITT/2024/011	Designed frontend UI components (HTML5/CSS3), structured client-side form validation scripts in app.js, designed database ER diagrams, and drafted project testing documentation
M A M Ahsan	ITT/2024/009	Built PHP backend architecture, configured MySQL PDO database connection scripts, implemented user auth/session guard systems, created admin routing pages, and authored database.sql schema.

6. Setup Instructions

How to import the database and run the project using XAMPP:

1. **Start Environment:** Install XAMPP and start Apache and MySQL services from the XAMPP Control Panel.
2. **Deploy Source Files:** Extract and copy the OMERO-GYM folder into your web root directory (e.g., C:\xampp\htdocs\OMERO-GYM).
3. **Import Database:** Open <http://localhost/phpmyadmin>, create/select the target database context, and import database.sql to initialize the 5 tables and default data catalog.
4. **Configure Database Connection:** Verify host, port, database name (omero_gym), and credentials in includes/db.php (default XAMPP setup: root with no password).
5. **Seed Admin Credentials:** Navigate to http://localhost/OMERO-GYM/admin/seed_admin.php in your browser once to register the initial administrator account.
6. **Application Access:**
 - Visit <http://localhost/omerogym/login.html> to log in as a member or administrator.
 - Open <http://localhost/OMERO-GYM/contact.php> to test whether contact messages are stored correctly in the database.

7. References

- [1] ". p. -. M. P. D. 2. [he PHP Group, <https://www.php.net/manual/en/function.password-hash.php>.
- [2] T. P. Group, PHP: password_hash - Manual," PHP Documentation, 2026. [Online], <https://www.php.net/manual/en/function.password-hash.php>.
- [3] T. P. Group, PHP: PDO Prepared Statements - Manual," PHP Documentation, 2026. [Online], <https://www.php.net/manual/en/pdo.prepared-statements.php>.
- [4] T. P. Group, PHP: Sessions - Manual," PHP Documentation, 2026. [Online]., <https://www.php.net/manual/en/book.session.php>.
- [5] M. D. N. (MDN), Fetch API - Web APIs," MDN Web Docs, 2026. [Online]., https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API.
- [6] A. Friends, XAMPP Documentation," Apache Friends, 2026. [Online], <https://www.apache.org/docs/>.